



Summer Internship-2026: SITER Academy, Norge

Git Version Control System

Introduction:

Git is the preferred Version Control System (VCS) for modern software development due to its distributed architecture, speed, robust branching and merging capabilities, and support for seamless collaboration. It addresses critical needs in development by tracking every change, enabling teamwork, and providing a safety net for experimentation.

Git enables developers to work **faster, more collaboratively, and more reliably** by providing a powerful, flexible system for tracking, managing, and merging code changes throughout a project's lifecycle.

Key Reasons to Use Git

1. Distributed Architecture

Git is a Distributed Version Control System (DVCS), meaning every developer has a complete copy (a local repository) of the project's entire history.

- **Offline Work:** Developers can commit changes, review history, and create branches even without a network connection.
- **Redundancy and Backup:** Since every local copy is a full backup, the project is highly resilient against server failures.
- **Speed:** Most operations (like committing or viewing history) are performed locally, making Git significantly **faster** than centralized VCS options

2. Powerful Branching and Merging

Git's ability to create and manage branches is one of its most important features, facilitating non-linear, parallel development.

- **Isolated Development:** Branches allow developers to work on new features, bug fixes, or experiments in isolation without affecting the stable main codebase.
- **Safe Experimentation:** If an experimental change fails, the branch can simply be discarded without any loss to the main project.
- **Easy Integration:** Git is optimized for merging changes between branches, making integration straightforward and resolving conflicts efficiently.

3. Collaboration and Workflow

Git is the foundation for collaborative platforms like [GitHub](#), GitLab, and Bitbucket, streamlining team workflows.

- **Parallel Work:** Multiple developers can work on the same project simultaneously, creating their own local branches and later integrating their work.
- **Code Review:** Features like Pull Requests (or Merge Requests) allow team members to review, discuss, and test changes before they are merged into the main branch, ensuring code quality.
- **Traceability:** Git keeps a detailed, secure history, noting **who** made **what** change, **when**, and **why** (via commit messages). This is crucial for debugging and auditing

4. Data Integrity and Efficiency

Git is designed to ensure the integrity of your code's history

- **Cryptographic Security:** It uses the **SHA-1** cryptographic hash function to uniquely identify and track every change, protecting the code from unnoticed corruption or tampering.
- **Snapshot-Based:** Unlike older VCSs that stored only the *differences* between versions, Git stores a complete snapshot of the entire repository's files at each commit, which is more efficient for historical viewing and retrieval.

Real World Examples

Git is the industry standard for version control, and its real-world use spans virtually every sector of technology, from massive enterprise systems to the world's most critical open-source projects.

Here are some real-world examples illustrating how Git is used across different scales and purposes:

1. Large-Scale Enterprise and Tech Companies

Major technology companies use Git, often combined with custom tooling, to manage massive, complex codebases and development teams scattered across the globe.

Company/Project	Use Case/Workflow	Git Feature Highlight
Linux Kernel	The original motivation for Git's creation. Managed by thousands of developers globally.	Distributed Model & Speed in handling millions of lines of code and hundreds of thousands of commits.
Microsoft	Uses Git for many major open-source projects (e.g., Visual Studio Code , TypeScript) and internally for many products.	Branching and Pull Requests on GitHub for collaborative development and code review.
Google	While Google uses a massive <i>monorepo</i> (a single giant repository) internally, Git principles and technology are heavily involved in their systems.	Version Tracking and custom tools built on Git concepts to manage code at an unprecedented scale.
Amazon	Manages its vast ecosystem using a micro-services architecture, where each service is often its own repository.	Micro-Repositories and integration with CI/CD pipelines for rapid, isolated deployments.
Financial/Defense	Companies in highly regulated industries use Git on private, self-hosted servers (e.g., GitLab or Bitbucket) for strict version control and compliance.	Security, Traceability, and Auditable History to comply with regulations.

2. Global Open-Source Projects

Git and its hosting platform, [GitHub](#), have become synonymous with open-source collaboration, enabling millions of developers to contribute to projects effortlessly.

- **React, Vue, Flutter (Frameworks):** These popular UI frameworks are developed entirely on public Git repositories. The collaborative process involves:
 - Developers forking the main repository.

- Creating a feature branch on their fork to make changes.
- Submitting a Pull Request for their changes to be reviewed, discussed, and merged by the maintainers.
- **Kubernetes (Container Orchestration):** A massive, complex project involving contributions from hundreds of companies and thousands of individuals. Git enables coordinated development and feature releases without chaos.
- **FreeCodeCamp, Mozilla Firefox, Android OS:** These projects rely on Git's distributed model to allow a massive, global community of volunteers to work asynchronously on bug fixes, new features, and documentation.

3. Beyond Software (Non-Code Uses)

Git's fundamental ability to track changes in text files extends its utility far beyond just source code.

Real-World Scenario	Git Feature Used	Benefit
Technical Documentation	Commits and History	Writers can track every change made to a user manual or API document, revert specific edits, and see who modified a particular section.
Website Content (CMS)	Branching (for staging)	A writer can draft a new blog post in a draft-branch , a reviewer can check it, and then it's merged into the production-branch for deployment.
Data Science / Machine Learning	Commits and Tags	Data scientists use Git to version control their Jupyter Notebooks , model training scripts, and configuration files, making their experiments reproducible .
Infrastructure as Code (IaC)	Pull Requests and Audit Log	DevOps teams version their configuration files (like Terraform or Ansible scripts) in Git, ensuring every change to the production infrastructure is reviewed and logged.

Git is not just a tool for professional developers; it's a foundational technology that powers collaborative work on digital projects of any kind, ensuring a clear history, safety, and an organized workflow.

Git Tools

The tools can be broken down into three main categories: the Core Commands (the foundation), Graphical User Interfaces (GUIs), and Ecosystem/Workflow Tools (for collaboration and advanced use).

1. The Core Command-Line Tools (CLI)

These are the fundamental commands that make up Git itself, which you execute directly in your terminal or command prompt.

Category	Key Commands	Purpose
Setup & Initializing	<code>git init</code>	Initializes a new Git repository.
	<code>git clone</code>	Creates a local copy of a remote repository.
Saving Changes	<code>git add</code>	Stages files for the next commit.
	<code>git commit</code>	Records the staged changes to the local repository history.
	<code>git status</code>	Shows the state of the working directory and staging area.
Branching & Merging	<code>git branch</code>	Lists, creates, or deletes branches.
	<code>git checkout</code> / <code>git switch</code>	Switches between branches or restores files.
	<code>git merge</code>	Combines changes from one branch into another.
Sharing & Syncing	<code>git remote</code>	Manages connections to remote repositories.
	<code>git push</code>	Uploads local branch commits to the remote repository.
	<code>git pull</code>	Fetches and merges changes from the remote repository.
Inspecting History	<code>git log</code>	Shows the commit history.
	<code>git diff</code>	Shows differences between commits, branches, or files.
Advanced Tools	<code>git rebase</code>	Rewrites commit history to maintain a linear history.
	<code>git reset</code> / <code>git revert</code>	Commands to undo changes in history.
	<code>git stash</code>	Temporarily saves changes that are not ready to be committed.

2. Graphical User Interface (GUI) Clients

Many developers prefer a visual interface over the command line for complex operations like merging, rebasing, and viewing history.

Tool Name	Key Features	Why it's Popular
GitHub Desktop	Simple interface, tailored for GitHub users, easy branch management.	Beginner-friendly and excellent for quick, daily tasks.
Sourcetree	Full Git-Flow support, detailed visual log of history, drag-and-drop actions.	Free and popular among teams using Atlassian products (Jira, Bitbucket).
GitKraken	Sleek, visual commit graph, built-in merge tool, cross-platform.	Highly visual and great for understanding complex branch history.
Tower	Powerful features, includes an in-app "Conflict Wizard," clean design (Mac & Windows).	Known for its ease of use and advanced functionality.
TortoiseGit	Integrates directly into the Windows File Explorer shell.	Excellent for Windows users who prefer right-click access to Git commands.
IDE Integrations	GitLens (VS Code), Git Integration (JetBrains IDEs)	Brings detailed Git functionality (blame, history) directly into your code editor.

3. Ecosystem & Collaboration Tools

These are external platforms and utilities that host your Git repositories and provide essential collaboration features around the core Git commands

Tool Name	Category	Primary Function
GitHub	Repository Host	The largest host for Git repositories, providing Pull Requests, Issues, Actions (CI/CD).
GitLab	Repository Host / DevOps Platform	Hosts repos and provides an integrated, end-to-end platform for the entire DevOps lifecycle (CI/CD, Monitoring, Security).
Bitbucket	Repository Host	Popular for teams using Jira and other Atlassian products, offers unlimited private repositories.
Jenkins / CircleCI / GitHub Actions	Continuous Integration/Deployment (CI/CD)	Tools that automatically build, test, and deploy code every time changes are pushed to a Git repository.
Git LFS (Large File Storage)	Utility	An extension to Git that handles large files (like design assets, videos, or big data) outside the core repository, which standard Git struggles with.

The tool you use depends on your preference: many experienced developers primarily use the Command Line Interface for speed and control, while others rely on a **GUI Client** or their **IDE** integration for a visual workflow.

In this learning section, we are focusing on Git and GitLab.

SSH Key in Git

An SSH key is an access credential used in the Secure Shell (SSH) protocol, which provides a secure, encrypted way to communicate between a client and a remote server over an unsecured network. You should use an SSH Key in Git (e.g., with platforms like [GitHub](#), GitLab, or Bitbucket) primarily for enhanced security and streamlined convenience when connecting to your remote repositories.

Once you set up your SSH key and add the public key to your Git host account, you no longer need to enter your username and password or PAT for every **git push**, **git pull**, or **git fetch** operation. The authentication happens automatically in the background.

Security Benefits

- **Stronger Authentication:** SSH keys use large, cryptographically complex key pairs (public and private keys) that are virtually impossible for an attacker to guess via brute force.
- **No Credentials Sent Over the Network:** Unlike password or PAT authentication (which is often done over HTTPS), your private key is never sent to the server. The key is only used locally to cryptographically sign the connection request, making it extremely difficult for credentials to be intercepted
- **Account-Wide Protection:** If an attacker somehow compromises an SSH key, they only gain access to the Git repositories associated with that key. If they compromise your password or PAT, they could gain access to your entire account settings and profile.

Passphrase Security Layer: Your private key is typically protected by an additional passphrase on your local machine. This means even if someone steals your private key file, they still can't use it without the passphrase.

Components of an SSH Key

SSH keys always come in a pair, utilizing public-key cryptography (asymmetric encryption):

- **Private Key:**
 - This key must remain secret and is stored securely on the client's local machine (your computer).
 - It's used to decrypt messages sent from the server, proving your identity.

You should never share your private key. It's often protected with an additional passphrase for an extra layer of security.

- **Public Key:**

- This key can be freely shared and is placed on any server you wish to access securely (typically in a file called **authorized_keys**).
- It's used by the server to encrypt a challenge message that can *only* be decrypted by the corresponding private key.

How SSH Key Authentication Works

1. **Client Request:** You (the client) attempt to connect to a remote server using the SSH protocol.
2. **Server Challenge:** The server checks its list of authorized public keys. If your public key is found, the server uses it to encrypt a random challenge message and sends it back to your client.
3. **Client Response:** Your local SSH client uses your private key to decrypt the challenge message. It then sends the correct decrypted response back to the server.
4. **Authentication:** If the server receives the correct response, it confirms that you possess the matching private key for the public key it has on file, and grants you secure access without needing a password.

SSH key authentication is considered more secure and often more convenient than traditional password-based authentication, as the key is much longer and more complex than a typical password, making it virtually impossible to crack with brute-force attacks.

Git Video Lecture Demo

You should use Git every day as a software developer because it is the industry-standard tool for version control (tracking and managing changes to code). It's essential for maintaining a historical record of your work, collaborating efficiently with others, and safely managing the evolution of your projects.

By using Git daily, you move from simply writing code to professionally managing code, ensuring your work is secure, traceable, and easily integrated with a team.

Outline of the Git Lecture (Theory + **Practical** Demo):

- Explain Why Git?
- Install Git
- Git Commands, GitLab
- Git for Projects (Demo)

[Click Here to Download Video Lectures](#)

[Click Here to Download PDF Presentation](#)

Assignment #1

- Git is your first assignment prior to start the Summer Internship 2026.
- You should install Git, create a Git profile to use as your Portfolio.
- Publish a dummy project with a documentation in your Git profile.
- Submit your work in the following folder by creating a new folder with your full name under '**Assignment 1 Submission**' section.

Submission Folder:

https://drive.google.com/drive/folders/1upqEnnKKSkg0bDvRk8lg0bW5j5Cy_hX?usp=sharing

- Your submission must include the technical documentation of your work (support with Screenshots of your activities), explanation of Git and it's implementation, challenges faced etc.
- Submission Deadline: 20th June 2026 12:00 PM (Indian Time)